Same Facts, Different Predictions? EquiSchema: Certified Auditing of Storage Invariance and Role Identifiability in Relational Learning

Anonymous Author(s)

Abstract. Relational models are typically evaluated on a single physical schema. Yet a storage refactoring can preserve the same logical facts while changing their distance in the graph. We introduce EquiSchema, an audit that combines executable instance-level rewrite certificates and task-integrity contracts with paired predictions, seed-normalized drift, and logical-context diagnostics. The audit treats storage invariance and sensitivity to distinct foreign-key roles as separate properties. In two generated domains, three certified rewrites reduce to three unique context states; LightGBM and a typed message-passing control recover the intended context-loss mechanisms. Raw role interventions and serialization controls distinguish semantic role keys, positional slots, and complete role collapse. A version-pinned source audit identifies a Relational Transformer (RT) role-collision risk without claiming an end-to-end RT failure. We also run a five-seed, budgeted GraphSAGE audit on RelBench's official `rel-f1/qualifying-position` task. Vertical decomposition lowers sampled fact Jaccard to .701, but normalized excess schema drift is .0157 with a 95% seed/race-cluster resampling interval of $[-.0721, .1236]$. The interval includes zero, and the ten-epoch models underperform a training-median reference. The real-data experiment demonstrates the audit protocol, but does not establish excess predictive drift or production-model robustness.

Keywords: Relational deep learning · Relational foundation models · Schema invariance · Foreign-key roles · Database normalization · Robustness auditing

1 Introduction

A relational database can have several physical representations of the same application-level facts. Vertical decomposition, denormalization, and foreign-key (FK) reification can change join paths or graph distances while preserving those facts. A relational deep learning (RDL) pipeline that samples a bounded number of physical hops may therefore retrieve different information for the same target entity. Evaluation on a single schema does not measure that sensitivity.

Semantic roles impose a different requirement. Buyer and seller may reference the same customer table, but exchanging them changes a transaction's meaning.

A representation that collapses these predicates can produce the same prediction even when the task requires a different one. We call these two properties *storage invariance* and *role identifiability*.

RelBench standardizes relational prediction tasks [14], while Castor established schema independence as a learning objective [11]. RDB2G-Bench studies graph-modeling choices [2], and equivalent-schema robustness also arises in Text-to-SQL [8]. EquiSchema complements these lines with executable instance-level equivalence checks, fixed task contracts, entity-paired predictions, seed baselines, and context diagnostics. The contribution is an audit, not a general theory of schema independence.

Contributions. We contribute (i) executable storage-rewrite certificates paired with task-integrity contracts; (ii) entity-paired drift, seed baselines, and logical-context diagnostics; (iii) role-sensitivity controls and a version-pinned representation audit; and (iv) reproducible controlled calibration plus a real-data GraphSAGE audit. The real-data audit can return an inconclusive result when schema drift cannot be separated from fitting noise. Our claims concern the executed instances and protocols, not universal schema independence.

2 Related Work and Novelty Boundary

2.1 Database equivalence and metamorphic testing

Hull formalized relative information capacity between relational schemata [7]. Our executable round-trip certificate is narrower: it checks equivalence for a specific task and instance. Methodologically, EquiSchema is also related to metamorphic testing: when a conventional output oracle is unavailable, one tests necessary relations between outputs under transformations [18]. Here the metamorphic relation is prediction stability only for certified storage rewrites; semantic role interventions deliberately require sensitivity instead.

2.2 Schema-independent relational learning

Picado et al. formalized schema independence for relational learning and showed that conventional learners may change behavior when the same information is represented through decomposition or composition [11]. Modern RDL adds bounded sampling, heterogeneous edge types, temporal censoring, and stochastic fitting, motivating prediction-level audits with context diagnostics.

2.3 Relational database benchmarks and graph construction

RelBench provides standardized predictive tasks with temporal splits and a reference heterogeneous graph neural network (GNN) pipeline [14]. RDB2G-Bench systematically varies relational-to-graph modeling and shows that representation choice can materially affect performance [2]. These studies show that modeling

Table 1. Positioning against the closest lines of work. A checkmark means the property is a central evaluated object, not merely possible in the implementation.

Work	Eq. cert.	Predictive RDL	Paired pred.	Role ID	Layer diag.
Castor [11]	✓	–	–	–	–
RDB2G-Bench [2]	–	✓	–	–	–
Same Data, Different Schemas [8]	✓†	–	–	–	–
FK-intervention study [3]	–	✓	–	–	✓
EquiSchema	✓	✓	✓	✓	✓

† Equivalent-schema construction for Text-to-SQL rather than a database round-trip certificate.

choices matter. They do not require an inverse certificate for representations of the same logical database instance, and paired prediction consistency for the same entities is not their focus.

Relational models. Relational foundation models (RFMs) extend this setting toward transfer. RelGNN introduces composite message passing for challenging relational structures [1], and RelGT uses transformer-style processing over relational graph structure [4]. The Relational Transformer (RT) moves toward zero-shot transfer across relational schemas [12]; RT-J scales pretraining and context-efficient prediction on real relational data [13]. RelArena-alpha provides a unified contemporary evaluation environment and strong flattening/model baselines including TabPFN-Rel [6].

Mechanistic work on synthetic relational pretraining has begun to use FK interventions to test whether learned predictors actually depend on relational structure [3]. That work asks whether pretraining induces relational computation. Given a predictive pipeline, EquiSchema tests whether its predictions remain stable under certified storage rewrites and respond to semantic role changes.

Adjacent robustness and task integrity. Kanchinadam et al. study large language model (LLM) Text-to-SQL robustness under equivalent schemas [8]. Query-generation consistency differs from paired predictive consistency and temporal context selection.

Relational Task Generation Language (RTGL) reports temporal inconsistencies in manually authored relational task SQL, including future-activity eligibility in RelBench's `driver-dnf` [10]. Because task leakage is orthogonal to storage invariance, we make task integrity an explicit precondition for external schema auditing.

3 Problem Formulation

Let $D \in \mathcal{D}(S)$ be a legal database instance under physical schema S. A predictive task τ specifies target entities, labels (when observed), one or more temporal cutoffs, and an evaluation metric. Let $A(D, \tau; \xi)$ be a complete prediction

procedure including preprocessing, neighborhood/context selection, fitting or in-context prediction, and randomness ξ. For target entity e, let $p_A(e \mid D, \tau; \xi)$ denote a score or predictive distribution.

3.1 Logical facts and certified storage equivalence

We use $\mathcal{F}(D)$ to denote a task-relevant logical fact interpretation rather than literal table bytes. A physical rewrite may introduce or remove tables while preserving $\mathcal{F}(D)$. To make this claim auditable, we require an executable certificate.

Definition 1 (Rewrite certificate). *A rewrite certificate is $C = (T, U, \phi, \Omega)$ where T maps instances of schema S to schema S', U is an executable inverse, ϕ maps target-entity identifiers from D to $T(D)$, and Ω is a set of admissibility preconditions (keys, functional dependencies, null semantics, row coverage, and temporal conditions). A rewrite is* certified for task τ *when, for all checked legal instances satisfying Ω,*

$$U(T(D)) \equiv_\tau D, \tag{1}$$

where $\equiv_\tau$ requires equality of task-visible logical facts, preservation of the target mapping ϕ, unchanged labels, and unchanged task cutoffs.

Certificates encode the inverse's admissibility assumptions. For N:1 denormalization these include the dimension functional dependency and retention of unused members. Verification compares typed keyed rows, or multisets where duplicates matter, and preserves null absence. Temporal joins must use information available at the task cutoff; a future-state snapshot is inadmissible. These conditions enter Ω and Algorithm 1.

Definition 2 (Task-integrity contract). *For temporal task τ, $B = (Q, I, H, \Gamma)$ records the target query, entity-eligibility rule, declared time horizon, and hidden-column policy. It is* time-safe *when eligibility at anchor t uses no information after t, labels use only the declared target window, and evaluated keys/labels/cutoffs remain unchanged across certified views. Autocomplete tasks use the analogous row-mask contract. Both C and B must pass before view drift is admissible.*

Task leakage/misalignment is therefore an L0 validity failure, not evidence for or against invariance.

3.2 Storage invariance

For a certificate C, define a prediction discrepancy $d(\cdot, \cdot)$, such as absolute probability change or hard-label disagreement. To compare schema drift and training noise under the same stochastic coupling, our primary view-drift estimand uses *independent* seeds:

$$\Delta^{\text{ind}}_{\text{view}}(T) = \mathbb{E}_{\xi \neq \xi'} \frac{1}{|E|} \sum_{e \in E} d(p_A(e \mid D; \xi), p_A(\phi(e) \mid T(D); \xi')) \,. \tag{2}$$

The same-schema baseline is

$$\Delta_{\text{seed}} = \mathbb{E}_{\xi \neq \xi'} \frac{1}{|E|} \sum_{e \in E} d(p_A(e \mid D; \xi), p_A(e \mid D; \xi')) \,, \tag{3}$$

and define excess schema drift (ESD) as

$$\text{ESD}^{\text{ind}}(T) = \Delta^{\text{ind}}_{\text{view}}(T) - \Delta_{\text{seed}}. \tag{4}$$

Thus both terms average over the same ordered independent-seed pairs. We additionally retain matched-seed $\Delta^{\text{pair}}_{\text{view}}$ as a sensitivity diagnostic because it removes one source of fitting noise, but it is not subtracted from an independently seeded baseline. Positive ESD means view changes add disagreement beyond refitting noise; ESD near zero means the audit does not detect excess drift. For the controlled hard-label results, 2,000 bootstrap replicates resample fitted seeds and test entities. The same seed weights are reused across views and both ESD terms; pairs with equal original seed IDs remain excluded. Thus the 20 ordered pairs are not treated as 20 independent fits. These intervals condition on the generated database and have limited resolution with five seeds.

For classification we also report area under the receiver operating characteristic curve (AUROC), calibration, and *worst-view performance*:

$$M_{\text{worst}} = \begin{cases} \min_{V \in \mathcal{V}} M(A, V), & \text{higher is better,} \\ \max_{V \in \mathcal{V}} M(A, V), & \text{lower is better.} \end{cases} \tag{5}$$

where $\mathcal{V}$ contains the original and certified views.

3.3 Role identifiability

Let r_1 and r_2 be two semantically distinct FK roles into the same parent table. A role intervention ρ swaps or otherwise changes these predicates while preserving the referenced entity values. Unlike a storage rewrite, ρ is *not* label preserving in general; it changes logical semantics.

Definition 3 (Role identifiability). *For a task where role changes can affect the target, a predictor is role identifiable on a pair $(x, \rho(x))$ if its response is directionally consistent with the verified role-conditioned target contrast. We report directional concordance and, when counterfactual target flips are available, the correct-flip rate.*

Lemma 1 (Role-collision limitation). *If an encoder maps a role-sensitive pair to the same representation, $\text{Enc}(x) = \text{Enc}(\rho(x))$, then any deterministic predictor $g \circ \text{Enc}$ returns the same prediction for the pair. Consequently it cannot be correct on both members whenever their Bayes-optimal decisions differ.*

Proof. Equality of encodings implies $g(\text{Enc}(x)) = g(\text{Enc}(\rho(x)))$. If the optimal labels differ, at least one equal decision is wrong. The statement is architectural and does not depend on the choice of g.

The lemma provides a diagnostic: a representation collision can be tested as an explanation before interpreting end-to-end performance.

4 The EquiSchema Audit

4.1 Certified rewrite family

We separate core storage rewrites from metadata perturbations.

R1: lossless vertical decomposition (1:1). For a keyed relation $R(K, A, B)$, create $R_A(K, A)$ and $R_B(K, B)$ with K unique in both. The inverse is a one-to-one join on K. This rewrite is useful because it can increase the physical distance between an entity and an attribute without changing the logical fact.

R2: N:1 denormalization. For fact relation $F(\ldots, K_D, \ldots)$ referencing dimension $D(K_D, Z)$, copy Z into F. The certificate requires $K_D \to Z$, explicit treatment of null FKs, and preservation of dimension rows needed for exact inversion. We recommend retaining a key-only dimension remainder or an auxiliary manifest for unused dimension members rather than pretending that an ordinary left join is always invertible.

R3: FK reification. Replace an FK column with a role-labelled link relation $L(K_F, r, K_P)$ and remove the direct edge. The inverse pivots or selects the unique (K_F, r) link back into the original FK column. Null FKs are represented by missing role links plus a null manifest where needed for exact round trip.

Metadata perturbations. Renaming a table or column while preserving relational algebra is *not* included in the core storage-invariance family when a model intentionally consumes schema text. Such tests measure metadata robustness, a separate property. This distinction avoids declaring meaningful schema semantics "irrelevant" by fiat.

4.2 Logical context and layerwise diagnosis

Let $N_A(e, D)$ be the set of logical fact identifiers actually made visible to the predictor for entity e after preprocessing/sampling. For a matched pair we compute logical-context coverage

$$\mathrm{Cov}(e; D, D') = \frac{|N_A(e, D) \cap \psi^{-1}(N_A(\phi(e), D'))|}{|N_A(e, D) \cup \psi^{-1}(N_A(\phi(e), D'))|}, \tag{6}$$

where ψ maps transformed records back to logical fact IDs; coverage is one when both sets are empty. This enables four diagnostic layers:

- **L0 – certificate failure:** the rewrite is not task-equivalent; no invariance claim is valid.
- **L1 – encoding collision:** two semantically distinct role configurations become the same representation (Lemma 1).
- **L2 – context-selection mismatch:** the views are equivalent, but bounded physical-hop/fanout retrieval exposes different logical facts; Cov < 1.

Algorithm 1 Certificate verification and paired-entity manifest construction

Require: Database D, task τ, rewrite T, inverse U, entity map ϕ, preconditions Ω
Ensure: Verified transformed database D' and pairing manifest P, or Reject

Validate keys, FKs, functional dependencies, null policy, and temporal assumptions in Ω
if any precondition fails **then**
    return Reject
end if
$D' \leftarrow T(D)$
$\widehat{D} \leftarrow U(D')$
Canonicalize every task-visible relation (types, key order, nulls, duplicate semantics)
if $\mathrm{hash}_\tau(\widehat{D}) \neq \mathrm{hash}_\tau(D)$ **then**
    return Reject
end if
Verify task contract B: cutoff(s), eligibility/history bounds, target window, hidden columns
Hash evaluated entity–time keys and labels; require equality across D and D'
Verify ϕ is one-to-one over evaluated target entities
if any task-integrity invariant fails **then**
    return Reject
end if
Emit pairing manifest $P = \{(e, \phi(e)) : e \in E_\tau\}$ together with logical-fact identifiers for context comparison
return $(D', P, \text{Verified})$

L3 – predictor/training instability: logical context is coupled (Cov $\approx$ 1), yet predictions still differ beyond seed noise.

The layers help distinguish an encoder collision from a sampling mismatch caused by a change in physical paths.

4.3 Mitigations

We evaluate explicit role-keyed channels and restoration of complete logical context. In the controlled domains, the latter is a hand-specified canonical flattened view. Training-time schema augmentation and a certificate-aware logical sampler remain *unevaluated extensions*. A logical sampler would need to improve the quality–latency/memory frontier over canonical materialization to justify its use.

5 System and Source-Code Audit

5.1 A concrete role-representation asymmetry

We audited version-pinned public code paths used to construct relational neighborhoods. In the RelBench heterogeneous graph builder, an FK edge type is

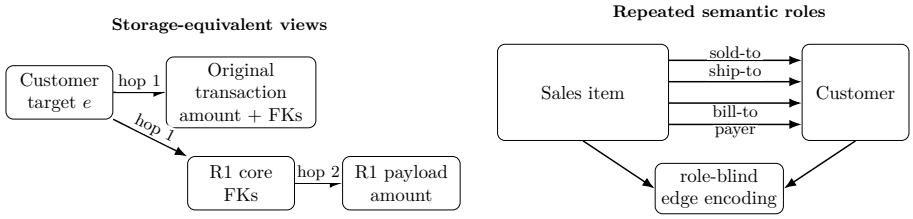


Fig. 1. Two complementary failure mechanisms. Left: a certified rewrite can preserve facts while changing physical distance, producing an L2 context-selection mismatch. Right: repeated FKs into the same parent table carry different semantics; collapsing their role identity creates an L1 representation collision.

constructed as the relation type `f2p_FK` (where `FK` is the originating FK column), with a correspondingly named reverse edge [17]. Thus two columns in the same child table that reference the same parent table remain distinct relation types.

In the examined RT/RT-J-compatible Rust preprocessing path, the input manifest maps FK column names to parent tables, but each serialized structural edge contains the neighboring node index, parent table identity/type, and timestamp; the edge structure does not store the originating FK-column identifier [15]. During construction, distinct FK columns that target the same parent table therefore contribute edges carrying the same parent-table type. This is an explicit L1 collision risk when downstream attention cannot reconstruct the original role from other evidence.

This source-code observation is not an end-to-end performance result: other features or table context may correlate with role. The representation hypothesis can be tested by adding or removing a role token while holding the rest of the system fixed.

5.2 Natural repeated-role schema: `rel-salt`

RelBench v2/v3 includes the enterprise `rel-salt` dataset. Its sales-item relation contains four customer FK roles (sold-to, ship-to, bill-to, and payer), all referencing `customer` [16]. The repeated roles arise naturally, making the dataset suitable for a role-identifiability audit.

Repeated roles alone do not establish a label effect. A task-level experiment must also show that role-conditioned information predicts the selected target under the official cutoff.

6 Controlled Experimental Calibration

The generated study *calibrates* the audit using known logical facts and certificates. Its sampler is intentionally simple, so the study does not show whether a production GNN or RFM fails on these rewrites.

6.1 Domains, certificates, and representation states

We generate two independent domains, each with 3,000 customers, 240 merchants, and 45,000 transactions. A transaction contains buyer, seller, and merchant FKs, amount, and time. The balanced target depends on an entity attribute plus buyer/seller-conditioned amount, count, and merchant-risk aggregates with opposing signs. Synth-B is heavier tailed than Synth-A.

We execute R1 (vertical decomposition), R2 (N:1 denormalization with an unused-dimension remainder), and R3 (role-labelled FK reification), run $U(T(D))$, canonicalize rows, and verify exact equality; SHA-256 round-trip hashes are emitted. Under the bounded physical-path sampler, however, several schema views intentionally induce the *same feature state*. We therefore do not count those rows as independent replications. State completeness below counts three transaction-contribution channels. Paired Jaccard additionally includes the entity attribute; merchant-risk IDs denote joined transaction contributions, not distinct merchant cells. The controlled states are

- S_{low}: structure only (coverage .33; R1_1, R3_1),
- S_{mid}: structure + amount (coverage .67; O_1, R3_2),
- S_{full}: structure + amount + merchant risk (coverage 1; O_2, R1_2, $\text{R2}_1/\text{R2}_2$, R3_3).

R1 therefore realizes $S_{\text{mid}} \to S_{\text{low}}$, R2 realizes $S_{\text{mid}} \to S_{\text{full}}$, and the chosen R3 stress realizes the reverse $S_{\text{full}} \to S_{\text{mid}}$. The latter two share the same unordered representation pair, so equality of their raw hard-disagreement rates is expected rather than evidence from two independent mechanisms.

6.2 Models and protocol

Canonical-flat LightGBM. LightGBM [9] receives the materialized relational aggregates plus the entity feature. At S_{full} this is an explicit canonical flattening baseline (hand specified, not an automated Deep Feature Synthesis implementation). We use 130 trees, learning rate .04, at most 15 leaves, depth at most 5, subsampling, and seeds $\{11, 23, 37, 51, 71\}$.

Typed message-passing neural network (TypedMPNN). The neural control applies separate buyer/seller linear message maps before a two-layer head. Each role message contains amount, structural count, and merchant-risk aggregates. We use AdamW, binary cross entropy, early stopping on validation AUROC, and the same seeds. No view-specific payload mask is applied; feature availability follows the physical-path traversal.

All views share entity splits. Preprocessing is fitted only on training entities in the complete original state and then frozen across views. The latent oracle retains its separate, fixed population transformation defining the data generator. Table 2 reports each *unique* controlled state once.

Table 2. Unique controlled representation states (AUROC, mean ± std over five seeds). Duplicate schema views mapping to the same state are listed rather than re-counted as independent evidence.

State	Views	Cov.	Synth-A		Synth-B	
			LGBM	MPNN	LGBM	MPNN
S_{low}	R1_1, R3_1	.33	0.845±0.000	0.848±0.006	0.839±0.001	0.794±0.032
S_{mid}	O_1, R3_2	.67	0.914±0.001	0.921±0.003	0.883±0.001	0.884±0.004
S_{full}	O_2, R1_2, $\text{R2}_1/\text{R2}_2$, R3_3	1.00	0.955±0.001	0.970±0.002	0.899±0.001	0.910±0.006

Table 3. Hard-label drift in percentage points using independent-seed coupling. ESD is $\Delta^{\text{ind}}_{\text{view}} - \Delta_{\text{seed}}$; brackets are seed/entity-bootstrap 95% resampling intervals.

Model	Dom.	Transition	Δ_{seed}	$\Delta^{\text{ind}}_{\text{view}}$	ESD [95% CI]
LightGBM	A	mid→low (R1)	1.87	20.41	18.54 [16.03,21.26]
	A	mid→full (R2)	1.87	11.42	9.56 [7.63,11.54]
	A	full→mid (R3)	2.09	11.42	9.33 [7.39,11.36]
	B	mid→low (R1)	2.62	14.42	11.80 [9.66,14.00]
	B	mid→full (R2)	2.62	8.58	5.96 [4.33,7.51]
	B	full→mid (R3)	2.96	8.58	5.62 [4.04,7.28]
TypedMPNN	A	mid→low (R1)	3.02	33.82	30.80 [19.89,39.97]
	A	mid→full (R2)	3.02	13.57	10.54 [8.46,12.62]
	A	full→mid (R3)	3.09	13.57	10.48 [8.53,12.51]
	B	mid→low (R1)	4.02	24.38	20.36 [11.15,28.04]
	B	mid→full (R2)	4.02	10.83	6.81 [5.09,8.76]
	B	full→mid (R3)	5.31	10.83	5.52 [3.75,7.42]

6.3 Seed-matched and independent-seed drift

Table 3 makes the primary comparison statistically symmetric: both $\Delta^{\text{ind}}_{\text{view}}$ and Δ_{seed} average the same 20 ordered independent-seed pairs. ESD intervals use the paired seed/entity bootstrap described above. The intervals exclude zero for every reported transition. R2 and R3 remain directionally different audits but share the same unordered $S_{\text{mid}}/S_{\text{full}}$ representation pair; they must not be interpreted as two independent replications.

6.4 Coverage sanity check

The sampler determines which logical components are reachable, so a monotone coverage curve is partly a consequence of its construction. Figure 2 uses the curve as a calibration check: on R1 with two-hop traversal, first-hop fanout varies the measured coverage while everything else is fixed. We plot matched-seed view disagreement separately for each generated domain, with seed/entity bootstrap intervals for that same paired estimand. The x-axis is paired contribution-fact Jaccard, rather than the fraction of full context. Disagreement falls toward zero as the coupled representations become identical at coverage one. That endpoint is expected by construction and checks the metric and sampler coupling.

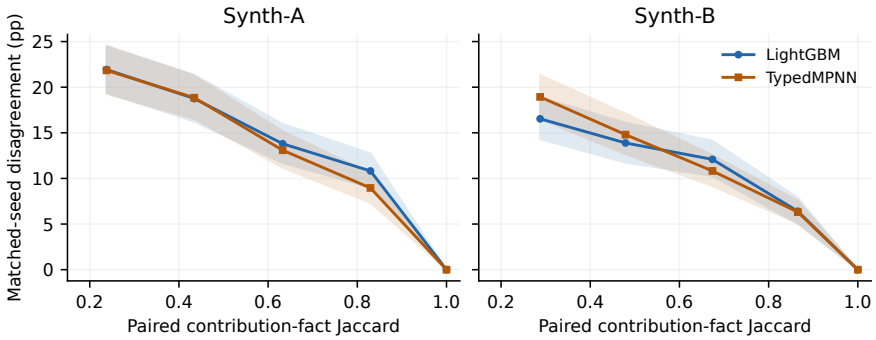


Fig. 2. Calibration of contribution-fact coverage. First-hop fanout under R1 changes paired Jaccard; bands are within-domain seed/entity-bootstrap 95% intervals for matched-seed disagreement. The zero endpoint at coverage one is a deterministic sanity check of the coupled representation, not external evidence of invariance.

Table 4. Role controls (%, five-seed mean). Swap concordance tests a semantic buyer/seller intervention; order drift tests a role-preserving physical slot permutation.

Model	Encoding	Swap concord.		Order drift	
		A	B	A	B
LightGBM	role-keyed	93.93	94.44	0.00	0.00
LightGBM	anonymous-slot	93.93	94.44	77.38	68.38
LightGBM	collapsed	0.00	0.00	0.00	0.00
TypedMPNN	role-keyed	98.36	97.24	0.00	0.00
TypedMPNN	anonymous-slot	98.36	97.24	77.20	71.00
TypedMPNN	collapsed	0.00	0.00	0.00	0.00

6.5 Role controls beyond the trivial collapsed baseline

A buyer/seller intervention swaps the raw transaction FKs and recomputes aggregates under frozen preprocessing, preserving referenced entities and non-role attributes. The collapsed control sums raw role contributions before fitting its own train-only preprocessing, preventing role-specific scales from reintroducing role information. The latent generator supplies the expected direction. A fully collapsed encoding is invariant by construction and therefore remains a negative control, not substantive evidence. We add an intermediate *anonymous-slot* control: buyer/seller channels stay separate but the encoder has no semantic role key and relies only on stable slot order. It responds to role swaps as strongly as the role-keyed model when ordering is stable, yet a role-preserving serialization-order permutation causes 68.4–77.4% hard-label disagreement. In contrast, the role-keyed loader canonicalizes by role label and is invariant to that physical ordering change.

6.6 What this calibration can and cannot establish

The audit detects the controlled loss or gain of logical context and distinguishes semantic role keys, anonymous positional slots, and complete role collapse. It

Table 5. Executed RelBench audit. NMAE is lower-is-better; drift is normalized by training-target standard deviation. Intervals are conditional seed/race-cluster resampling intervals.

Quantity	Value
Original GraphSAGE NMAE, mean ± std	0.9000 ± 0.0230
R1 GraphSAGE NMAE, mean ± std	0.8864 ± 0.0297
Training-median reference NMAE	0.8107
Mean paired fact Jaccard	0.7006
Same-schema independent-seed drift	0.0745
Cross-view independent-seed drift	0.0902
ESD [95% interval]	$0.0157\ [-0.0721, 0.1236]$

does *not* establish that R1–R3 have distinct end-to-end effects in production systems: in this calibration they reduce to three feature states, and two stress directions share the same unordered state pair. Likewise, the role source audit remains a hypothesis until an RT-family behavioral ablation is executed. The canonical full-context state evaluates one simple mitigation; training-time schema augmentation and a certificate-aware sampler remain untested. L0 is exercised by the artifact task-contract linter and the runtime admission checks in Sec. 7; this paper does not intentionally manufacture an L3 failure.

7 Real-Data Audit: `rel-f1`

We executed the official autocomplete regression task `qualifying-position`, using pinned RelBench code and data manifests [17,16]. The task uses normalized mean absolute error (NMAE). Its 9,815 target rows comprise 2,228 training, 1,854 validation, and 5,733 test entities. Target keys, timestamps, labels, and the hidden `qualifying.position` column are fixed across views. R1 decomposes the 26,080-row `results` relation into FK/time core and attribute payload tables, with copied time metadata and an exact typed round trip. Runtime checks reject sampled nodes after the target cutoff, including future race outcomes that could expose a qualifying-position proxy through `results.grid`.

Budgeted reference pipeline. We use RelBench's heterogeneous GraphSAGE implementation [5]: two layers, 32 channels, fanouts [32,16], batch size 512, Adam at .005, and ten epochs with validation-NMAE checkpoint selection. Each view is retrained using seeds $\{11, 23, 37, 51, 71\}$. This reduced configuration is an audit case study, not a leaderboard reproduction. Graph feature materialization follows the transductive autocomplete reference pipeline on the masked full database; target labels are excluded. Evaluation sampling is fixed across fits. Native NMAE and regression drift both use the training-target standard deviation, 6.6389. Resampling uses shared seed weights and race clusters; five seeds and shared drivers across races limit inferential precision.

Outcome and canonical control. Table 5 shows context mismatch but no resolved excess predictive drift: the ESD interval includes zero. Both trained configurations underperform the training-median reference, so this result cannot establish invariance of well-fitted production models. Reconstructing the original schema restores its keyed rows, edge topology, and field mapping. Actual CPU inference using the original checkpoints differs from GPU predictions by at most 1.13×10^{-6} in mean absolute prediction per seed (maximum individual difference 1.74×10^{-4}), consistent with numerical roundoff. This oracle inverse control is not an automated logical sampler or a separate L3 identification experiment. Measured function execution, including dependency installation, took 313 seconds; the experiment process peaked at 3.86 GiB RAM and 1.03 GiB allocated GPU memory.

8 Discussion and Limitations

The controlled calibration detects known context changes and separates semantic role keys from positional or collapsed encodings. These results do not establish widespread production failures. On the real task, measurable context loss coexists with an ESD interval spanning zero and weak predictive performance under the fixed training budget. The audit can report this inconclusive outcome; it does not show that physical refactoring is harmless.

External evidence is limited to one database, task, model configuration, and rewrite. Five fitted seeds provide limited uncertainty resolution; shared entities and relational dependencies further constrain resampling interpretations. Controlled intervals condition on two generated databases, and their contribution-level coverage differs from base-table fact coverage on RelBench. The RT source audit establishes a collision risk only; an RT behavioral ablation and a naturally role-sensitive `rel-salt` task remain future work. R2 requires its unused-dimension remainder, and metadata renaming remains outside core invariance when schema text is consumed. Neither training-time schema augmentation nor a certificate-aware logical sampler is evaluated, and no L3 effect-size claim is made.

9 Reproducibility and Artifact

The anonymized supplementary artifact contains executable R1–R3/SHA-256 certificates, five-seed runs, independent-seed/bootstrap analysis, role controls, ablation comma-separated-value (CSV) files, a task-contract linter, figures, and a pinned RelBench runbook. The real-data package includes ten checkpoints, ordered per-target predictions, canonical CPU outputs, version locks, task and rewrite hashes, sampler settings, and cost-envelope receipts. Tables and plots are regenerated from the saved outputs.

10 Conclusion

EquiSchema combines instance-level rewrite certificates, fixed task contracts, paired predictions, seed baselines, and context diagnostics. Controlled calibration recovers intended context and role mechanisms. A version-pinned RelBench run measures context mismatch but does not resolve predictive drift beyond seed noise under its limited training budget. The result separates a measured change in context from a predictive effect that the experiment cannot resolve.

Declarations

Data availability. Controlled data are generated by the reproduction code. Public RelBench data and task manifests are identified by pinned revisions in the artifact.

Ethics. This study uses generated and public benchmark data; no new human-participant data were collected.

Author contributions. Individual CRediT roles are withheld in this anonymous manuscript.

Funding. This research received no external funding.

Conflict of interest. The authors declare no conflict of interest.

References

1. Chen, T., Kanatsoulis, C., Leskovec, J.: RelGNN: Composite message passing for relational deep learning. In: Proceedings of the 42nd International Conference on Machine Learning. vol. 267, pp. 8296–8312. PMLR (2025)
2. Choi, D., Kim, S., Kim, J., et al.: RDB2G-Bench: A comprehensive benchmark for automatic graph modeling of relational databases. In: Advances in Neural Information Processing Systems (2025). https://doi.org/10.52202/085713-4078, datasets and Benchmarks Track
3. Dubey, S., Bouadi, M., Bouarour, N., et al.: When does synthetic relational data teach models to use relations? tracing predictive structure from pretraining data to model behavior. arXiv preprint arXiv:2610.03057 (2026)
4. Dwivedi, V.P., Jaladi, S., Shen, Y., et al.: Relational graph transformer. In: International Conference on Learning Representations (2026)
5. Hamilton, W.L., Ying, R., Leskovec, J.: Inductive representation learning on large graphs. In: Advances in Neural Information Processing Systems (2017)
6. Hayler, A., Floge, K., Arazi, A., Ranjan, R., Leskovec, J., et al.: Advancing open and reproducible relational learning: Relarena-alpha, tabpfn-rel and rpi. arXiv preprint arXiv:2608.16319 (2026)
7. Hull, R.: Relative information capacity of simple relational database schemata. SIAM Journal on Computing **15**(3), 856–886 (1986). https://doi.org/10.1137/0215061

8. Kanchinadam, N., Menachery, A., Deshpande, A.: Same data, different schemas: Robustness of llm-based text-to-sql. In: Ninth International Workshop on Exploiting Artificial Intelligence Techniques for Data Management (2026). https://doi.org/10.1145/3814940.3815329
9. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y.: LightGBM: A highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems (2017)
10. Kolesnichenko, O., Peleška, J., Šír, G.: Relational task generation language: A declarative specification framework for relational deep learning. arXiv preprint arXiv:2609.01292 (2026)
11. Picado, J., Termehchy, A., Fern, A., Ataei, P.: Schema independent relational learning. In: Proceedings of the 2017 ACM International Conference on Management of Data. pp. 205–218 (2017). https://doi.org/10.1145/3035918.3035923
12. Ranjan, R., Hudovernik, V., Znidar, M., et al.: Relational transformer: Toward zero-shot foundation models for relational data. In: International Conference on Learning Representations (2026)
13. Ranjan, R., Kothapalli, V., Agarwal, H., et al.: RT-J: Large-scale pretraining of relational transformers for context-efficient predictions. In: Advances in Neural Information Processing Systems (2026)
14. Robinson, J., Ranjan, R., Hu, W., et al.: RelBench: A benchmark for deep learning on relational databases. In: Advances in Neural Information Processing Systems (2024)
15. Stanford STAR Project: Relational Transformer source code: Rust preprocessing path (2026), GitHub repository stanford-star/relational-transformer, commit def35f92bf890741c4be41c4c315886365ee5ffb; accessed 26 October 2026
16. Stanford STAR Project: RelBench public data and task manifests for rel-f1 and rel-salt (2026), Hugging Face datasets; rel-f1 manifest revision d500f4d5eb581b6d768a1f5ac5a20db14d61c280 and rel-salt revision e90eab0ab2176a947d315d4cb4115a06eccf62fe; accessed 26 October 2026
17. Stanford STAR Project: RelBench source code: Heterogeneous graph construction (2026), GitHub repository stanford-star/relbench, commit 91c1f6420a03feb61c279aaeb194d7268435f278; accessed 26 October 2026
18. Xie, X., Ho, J.W.K., Murphy, C., Kaiser, G., Xu, B., Chen, T.Y.: Testing and validating machine learning classifiers by metamorphic testing. Journal of Systems and Software **84**(4), 544–558 (2011). https://doi.org/10.1016/j.jss.2010.11.920